

New upper and lower bounds on covering codes $K_q(n, R)$ for alphabets of size five to fifteen

Mark Marosi*

August 2026

Abstract

We improve both ends of the standard tables of bounds on $K_q(n, R)$, the minimum cardinality of a q -ary code of length n with covering radius R , in the range $5 \leq q \leq 15$. On the upper-bound side we give eighteen improved bounds, obtained by an engineered focused local search seeded with structural constructions; to our knowledge these are the first improvements to any upper bound on $K_q(n, R)$ with $q \geq 5$ since the 2011 freeze of Kéri’s tables, and they include reductions of up to 23% ($K_6(8, 4) \leq 166$) below bounds that had stood for decades. On the lower-bound side we give seven improved bounds for $6 \leq q \leq 10$, obtained from an exact-arithmetic reimplementation of the Gijswijt–Polak semidefinite programming hierarchy, whose published results cover only $q \leq 5$; every new lower bound is backed by a rational dual certificate that a standalone, dependency-free checker verifies in exact arithmetic, so the bounds do not rest on floating-point computation. One cell is improved from both sides: $441 \leq K_6(10, 4) \leq 2784$, against the previous 417–2952. Beyond the bounds themselves we report the algorithm engineering that produced them — a shared-sphere incremental evaluation that speeds up the local search 6–10 $\times$, structural solvers over cosets of linear codes and translation quotients, a measured portfolio policy, and a controlled comparison of CPU and GPU implementations — together with the exhaustive, redundant verification protocol. All eighteen codes and all seven certificates are provided in machine-readable form with standalone verifiers.

1 Introduction

Let $\mathbb{Z}_q^n$ denote the Hamming space of q -ary words of length n . A code $C \subseteq \mathbb{Z}_q^n$ has *covering radius* R if every word of $\mathbb{Z}_q^n$ is within Hamming distance R of some codeword. The function

$$K_q(n, R) = \min\{|C| : C \subseteq \mathbb{Z}_q^n \text{ has covering radius } \leq R\}$$

is a central quantity of covering-code theory [1]; the binary case $K_2(n, 1)$ includes the domination number of the hypercube, and the ternary case $K_3(n, 1)$ is the classical football pool problem.

Upper bounds on $K_q(n, R)$ are established by explicit constructions, lower bounds by combinatorial or convex-relaxation arguments, and the state of the art is recorded in the tables

*Department of Measurement and Information Systems, Budapest University of Technology and Economics (BME), Budapest, Hungary. email: marosi@mit.bme.hu. The search software, the verification protocol, the constructions, and this manuscript were produced by the AI system Claude (Anthropic), operating autonomously on a single NVIDIA GH200 node; the author’s contributions were the initiating idea, methodological guidance, and the computational resources. See the Acknowledgments.

of Kéri [2], which subsume and extend the earlier tables of Cohen, Honkala, Litsyn and Lobstein [1]. The last update to Kéri’s tables was made in November 2011. Recent activity has concentrated on lower bounds for small alphabets: Wu and Chen [4] improved binary lower bounds via congruence arguments, and Gijswijt and Polak [3] obtained many new lower bounds for $q \leq 5$ from a semidefinite programming hierarchy. For alphabets $q \geq 5$ we are not aware of any published improvement on either side since the 2011 freeze; a systematic search of the literature (arXiv, DBLP, OpenAlex, Lobstein’s covering-code bibliography, and the publication lists of the authors active in this area) found none.

This paper improves both sides in that range: eighteen upper bounds for $5 \leq q \leq 15$ (Table 1) and seven lower bounds for $6 \leq q \leq 10$ (Table 2).

Two structural observations explain where the improvements come from. On the upper-bound side, Kéri’s source annotations show that for moderate q and $R \geq 3$ most tabulated bounds are *inherited arithmetic* — direct sums

$$K_q(n_1 + n_2, R_1 + R_2) \leq K_q(n_1, R_1) \cdot K_q(n_2, R_2), \quad (1)$$

the special case $K_q(n + 1, R) \leq q K_q(n, R)$, the monotonicity $K_q(n + 1, R + 1) \leq K_q(n, R)$, and product rules over alphabet factorizations — cells in which no search was ever run. Those cells fall furthest (up to 23%). But the engineered search of Section 3 also shaves several cells whose incumbents came from dedicated search or codes in the 2000s (the cells with keys **n**, **q**, **z** in Table 1), by 1–6%. On the lower-bound side, the size of the symmetry-reduced Gijswijt–Polak SDP depends only on n and not on q , so their $q \leq 5$ boundary was one of compute and numerical precision rather than of model size; carrying the same relaxation into $6 \leq q \leq 10$ with exact certification yields improvements precisely on the cells whose incumbent lower bounds are weak relative to the sphere-covering bound (Section 5).

The results were produced autonomously by an AI system on a single GH200 node over roughly one week; we describe the methodology — including a frozen-rules competition between independently developed solvers, used to select the production algorithm — in Sections 3 and 4, and the verification protocol, which we regard as the load-bearing part of any record claim, in Section 6.

2 Results

Table 1 lists the eighteen new upper bounds with the construction that produced the previous bound, and Table 2 the seven new lower bounds. Three points deserve note.

First, the cell $K_6(10, 4)$ is improved from both sides, from 417–2952 to 441–2784. Second, none of the record cells’ searches is known to be exhausted: the three highest-redundancy cells ($K_6(10, 4)$, $K_6(10, 5)$, $K_7(9, 4)$) never stalled in any run — each stop was a scheduling decision, and successive sessions took $K_7(9, 4)$ from 1843 to 1743, 1600, and now 1479 — so the marked bounds should be read as a progress report on cells that had simply never been searched. Third, the seven lower-bound improvements land exactly where the incumbents are weakest: every improved cell carries Kéri key **y** or **s** (bounds within 1–4% of sphere-covering), and no cell with key **m** (Haas–Halupczok–Schlage–Puchta 2009, up to 52% above sphere-covering) is improved — on those cells the triple-distribution SDP at this level of the hierarchy is simply weaker than the best combinatorial arguments at $q \geq 6$ (Section 5.3).

Remark 1. *An audit of the tables of [2] against their own derivation rules found one internal gap: the tabulated $K_{17}(7, 2) \leq 252735$ is weaker than $K_{17}(7, 2) \leq 17 \cdot K_{17}(6, 2) = 17 \cdot 14424 =$*

cell	previous bounds	new UB	Δ	$\Delta\%$	key	previous UB via
$K_5(11, 5)$	103–625	602	−23	3.7%	d	Bhandari–Durairajan 1996
$K_6(7, 3)$	70–246	227	−19	7.7%	e	$K_q(n+1, R) \leq q K_q(n, R)$
$K_6(8, 3)$	246–1080	1005	−75	6.9%	f	direct sum
$K_6(8, 4)$	46–216	166	−50	23.1%	e	$K_q(n+1, R) \leq q K_q(n, R)$
$K_6(9, 4)$	136–738	660	−78	10.6%	f	direct sum
$K_6(9, 5)$	32–144	119	−25	17.4%	j	alphabet product rule
$K_6(10, 4)$	441 –2952	2781 [↓]	−171	5.8%	f	direct sum
$K_6(10, 5)$	83–615	486 [↓]	−129	21.0%	f	direct sum
$K_7(8, 4)$	76–343	316	−27	7.9%	c	monotonicity $K_q(n+1, R+1) \leq K_q(n, R)$
$K_7(9, 4)$	264–1843	1475 [↓]	−368	20.0%	f	direct sum
$K_8(6, 4)$	15–20	19 [↓]	−1	5.0%	n	Kéri–Östergård 2005
$K_8(8, 4)$	118–512	505	−7	1.4%	c	monotonicity $K_q(n+1, R+1) \leq K_q(n, R)$
$K_8(8, 5)$	29–90	85 [↓]	−5	5.6%	z	Kéri 2009
$K_8(10, 6)$	58–342	341 [↓]	−1	0.3%	n	Kéri–Östergård 2005
$K_{12}(6, 4)$	30–41	39 [↓]	−2	4.9%	q	Rivas Soriano 2005–2008
$K_{13}(6, 4)$	35–46	45 [↓]	−1	2.2%	q	Rivas Soriano 2005–2008
$K_{14}(6, 4)$	40–52	50 [↓]	−2	3.8%	q	Rivas Soriano 2005–2008
$K_{15}(6, 4)$	46–59	57 [↓]	−2	3.4%	c	monotonicity $K_q(n+1, R+1) \leq K_q(n, R)$

Table 1: New upper bounds. “Previous bounds” are the lower–upper pairs of Kéri’s tables [2] (a bold lower bound is this paper’s, from Table 2); “key” is the source annotation of the previous upper bound in [2], expanded in the last column. A [↓] marks bounds at which our descent had not yet stalled when this version was prepared: these are snapshots, not walls, and we expect them to improve further. All other bounds resisted a dedicated attack at $M - 1$.

cell	sphere bound	previous LB (key)	new LB	SDP value ^{1/3}	known UB
$K_6(9, 3)$	881	921 (y)	924	923.9804	4752
$K_6(10, 4)$	411	417 (s)	441	440.2854	2952
$K_7(8, 3)$	439	457 (y)	471	470.3230	2337
$K_8(8, 3)$	813	829 (y)	855	854.5627	4096
$K_8(9, 4)$	403	409 (s)	428	427.4131	2944
$K_9(9, 4)$	690	703 (s)	718	717.8794	5103
$K_{10}(9, 4)$	1123	1130 (s)	1145	1144.1984	8500

Table 2: New lower bounds, all with exact rational certificates. “Sphere bound” is $\lceil q^n/|B_R| \rceil$; “previous LB” is Kéri’s table [2] with its source key (y = improved sphere-covering bound, s = Lang–Quistorff–Schneider 2006); “SDP value^{1/3}” is the cube root of the certified rational value of the semidefinite bound of [3] (which bounds $K_q(n, R)^3$).

245208, which follows from the tabulated $K_{17}(6, 2) \leq 14424$ and the standard rule $K_q(n+1, R) \leq q K_q(n, R)$. We record this as an observation; it involves no new construction.

3 Upper bounds: the search

3.1 Focused local search

We search for a code of prescribed size M minimizing the number of uncovered words. The state is a multiset of M codewords together with the coverage count $\text{cnt}(w)$ of every $w \in \mathbb{Z}_q^n$; a move changes one coordinate of one codeword. The move update is cheap because of the following observation.

Lemma 1. *Let c' be obtained from c by setting coordinate p to $v \neq c_p$. Then the set of words leaving the radius- R ball of c and the set of words entering the ball of c' are both indexed by the words at distance exactly R from c in the coordinates other than p ; consequently a move updates exactly $2S$ coverage counters with $S = \binom{n-1}{R}(q-1)^R$, the two sets are disjoint, and distinct counter updates address distinct words.*

Proof. Since $d(w, c') = d(w, c) + [w_p = c_p] - [w_p = v]$, a word can leave the ball of c only if $d(w, c) = R$ and $w_p = c_p$, and can enter the ball of c' only if $d(w, c') = R$ and $w_p = v$; writing such a word as its restriction to the coordinates $\neq p$ (at distance exactly R from the restriction of c) plus its value at p gives the stated bijections, disjointness ($w_p = c_p$ versus $w_p = v$), and injectivity. $\square$

The search itself is a focused local search in the WalkSAT lineage, close in spirit to the tabu searches that produced much of the original tables [6, 5]: pick a random uncovered word u ; collect as candidate moves every codeword at distance exactly $R + 1$ from u nudged one coordinate towards u (if no codeword is that close, the codewords at minimum distance from u are used); evaluate all candidates exactly and in parallel; commit a best candidate, ties broken uniformly at random. On stagnation a codeword is teleported onto an uncovered word. Parameter studies on $K_6(6, 3)$ showed that *candidate breadth dominates*: evaluating all candidate moves exactly outperformed sampling 24 or 64 of them (final uncovered counts 0 versus 82 and 18), while tabu tenures and WalkSAT-style noise were harmful on these instances. Simulated annealing sustained a $33\times$ higher move rate and still lost to the focused search — move quality beats move count here.

For each cell we first rebuilt a code of the incumbent size from the same construction that produced the incumbent bound (e.g., for $K_6(10, 4)$ the direct sum (1) of an optimal $K_6(4, 1)$ code of size 72 and a $K_6(6, 3)$ code of size 41, both found by direct search and verified). Descent then proceeds by remove-and-repair: delete the codeword whose private coverage (words covered by no other codeword) is smallest, and repair the deficit by local search; on success, repeat at $M - 1$. Searching the small factors and re-composing (a “factor attack”) was decisive for seeding; the improvements themselves come from the descent breaking out of the product structure.

3.2 Algorithm engineering

The search described above spends essentially all of its time in one place, and saying exactly where turns out to determine what is worth optimising.

Profile

Instrumenting each phase of the solver with `CLOCK_THREAD_CPUTIME_ID` on three representative cells — $K_6(6, 3)$ at $M = 41$ ($q^n = 46\,656$), $K_6(8, 4)$ at $M = 169$ ($q^n = 1\,679\,616$) and $K_8(9, 4)$ at $M = 2944$ ($q^n = 134\,217\,728$) — gives candidate evaluation 92.9%, 98.5% and 99.6% of the run time respectively. Committing a move takes 3.3%, 0.8% and 0.4%; choosing the uncovered word, computing the M Hamming distances and breaking ties together account for under 2% everywhere. The cost of one evaluated sphere pattern grows from 1.9 ns to 6.0 ns to 7.4 ns as the counter array grows from 93 kB (L2-resident) to 3.4 MB (L3) to 268 MB (DRAM): the walk is a strided gather and its throughput is set by the working set, so reducing *bytes touched* pays as well as reducing *patterns walked*.

Sharing one sphere among the moves of a codeword

Lemma 1 localises both the leaving and the entering set of a move, and a closer reading shows that the leaving set depends on the moved coordinate only through a membership predicate. Write $D(w) = \{j : w_j \neq c_j\}$ and let $c^{(p)}$ denote c with coordinate p replaced by v .

Lemma 2. *w leaves the ball of c under the move at p if and only if $|D(w)| = R$ and $p \notin D(w)$; and w enters the ball of $c^{(p)}$ if and only if $|D(w)| = R + 1$, $p \in D(w)$ and $w_p = v$.*

Consequently, putting

$$T = \#\{w : |D(w)| = R, \text{cnt}(w) = 1\}, \quad H_j = \#\{w : |D(w)| = R, \text{cnt}(w) = 1, j \in D(w)\},$$

the number of words that the move at p newly uncovers equals $T - H_p$ for *every* p at once. A single enumeration of the full distance- R sphere, of size $\binom{n}{R}(q-1)^R$, therefore replaces the $R+1$ separate enumerations of $\binom{n-1}{R}(q-1)^R$ patterns each, a saving of $(R+1)(n-R)/n$ (2.00, 2.50 and 2.78 on the three cells). The updates to T and H are executed only when $\text{cnt}(w) = 1$, which on the loose cells this method targets happens for 0.4% of words at $M = 169$ and 0.07% at $M = 216$ in $K_6(8, 4)$, so the inner loop reduces to one load and one compare — half the memory traffic per pattern of the original.

The entering side from the uncovered words

The entering side does not share in the same way: enumerating the distance- $(R+1)$ sphere once costs $(q-1)n/(R+1)^2$ times the per-position walks, which is worse for these parameters. But the entering side counts only words with $\text{cnt}(w) = 0$, and by Lemma 2 membership of the entering set is decided in $O(n)$ from the digits of the word. Maintaining the set U of uncovered words explicitly therefore replaces $(R+1)\binom{n-1}{R}(q-1)^R$ counter reads by $O(n|U|)$ work, which is profitable exactly when

$$n|U| < (R+1)\binom{n-1}{R}(q-1)^R. \quad (2)$$

The right-hand side is 5 000, 109 375 and 840 350 on the three cells while the mean $|U|$ during a run is 58, 30 and $\approx 10^4$, so (2) holds by two to four orders of magnitude for the whole useful life of the search. The solver tests (2) each iteration and falls back to the sphere walk when it fails, so it is never worse. U is kept as an append-ordered array of (index, digit vector) pairs together with a presence bitset over $\mathbb{Z}_q^n$; commits append the words they uncover, stale entries are removed by an amortised compaction, and no q^n -sized index array is needed.

Together, the counter reads per candidate codeword fall from $2(R+1)\binom{n-1}{R}(q-1)^R$ to $\binom{n}{R}(q-1)^R$, a factor of 4.00, 5.00 and 5.56.

A wider neighbourhood for free

$T - H_p$ does not depend on the value v written into coordinate p , and the entering counts read off U are naturally indexed by the pair (p, v) at no extra cost. One sphere enumeration plus one pass over U therefore yields the exact objective change of *all* $n(q-1)$ single-coordinate moves of a codeword, not only the $R+1$ that move it towards u . Since the parameter study of Section 3.1 found candidate breadth to be the dominant tuning knob, widening the neighbourhood from $R+1$ to $n(q-1)$ moves per codeword — a factor of 7.5 to 11 here — is the natural way to spend the saving.

Narrower counters on the read path

Coverage counters are 16-bit because M may exceed 255, but evaluation only ever asks whether a counter is 0 or 1. A side array $\text{st}(w) = \min(\text{cnt}(w), 2)$ carries everything the read path needs in one byte or in two bits, while the exact counters stay for the write path; the extra write costs nothing measurable because commits are under 1% of the time. On $K_8(9, 4)$ the two-bit form shrinks the read path from 268 MB to 33.5 MB, bringing it inside the 117 MB last-level cache.

Correctness

Every one of these changes is an implementation change: it computes the same objective differences as the reference solver and must therefore follow the same search trajectory from the same seed. That is verified bit-for-bit over 24 configurations (q from 2 to 7, $R = 0$, $R = n - 1$, sizes above and below the optimum) $\times$ 9 variants, and separately the number of uncovered words the solver reports is required to equal the number the standalone verifier computes from the written code file — including the runs that end at a nonzero count.

Measurements

The host was shared with a production sweep for the whole measurement session, so wall clock is meaningless and every figure below is the solver’s own `CLOCK_PROCESS_CPUTIME_ID` reading, with the run order of the whole job list randomised. Two experiments were run. In the first (Table 3), every variant was given a fixed iteration budget; since they all follow the same trajectory this is a fully controlled comparison with no search-quality confound. In the second (Table 4), each cell was started from a fixed code file and each configuration was run with 20 seeds to a fixed target uncovered count, recording the CPU time at which the target was first reached.

After these changes the profile has moved: candidate evaluation falls to 72.8%, 81.7% and 98.2% of the run time, and committing a move — which concerns a single (p, v) pair and so cannot use the shared identity — rises to 19.2%, 14.5% and 1.6%.

What did not help

Several plausible ideas were implemented and measured, and did not work; we record them because the reasons are structural rather than accidental.

variant	$K_6(6, 3)$ 20 000 it	$K_6(8, 4)$ 600 it	$K_8(9, 4)$ 50 it
reference implementation	1.00×	1.00×	1.00×
peeled walk, hoisted offsets	1.35×	1.29×	1.13×
+ 8-bit state array	1.28×	1.65×	1.65×
+ 2-bit state array	0.82×	1.34×	2.44×
shared sphere, 8-bit state	1.32×	1.95×	1.95×
+ uncovered-word list	2.21×	5.57×	6.06×
with 2-bit state	2.14×	6.32×	9.86×

Table 3: Speedup in CPU time at a fixed iteration budget, median of 9 runs (6 on the largest cell). Every row performs the identical search: the final uncovered counts agree exactly, seed by seed, across all rows. The interquartile range is 0.9%, 2.2% and 5.6% of the median.

cell	M	target	speedup, identical search	with wide neighbourhood
$K_6(6, 3)$	41	$ U \leq 30$	$2.03\times [1.99, 2.13]$	$0.28\times$
$K_6(8, 4)$	169	$ U \leq 20$	$6.12\times [6.06, 6.62]$	$8.03\times [5.75, 11.02]$
$K_8(9, 4)$	2944	$ U \leq 10\,800$	$8.07\times [6.19, 10.72]$	$11.04\times [8.06, 12.17]$

Table 4: Median per-seed ratio of CPU time to target over 20 seeds, with a percentile-bootstrap 95% confidence interval. In the fourth column the optimised solver reached the target at exactly the same iteration as the reference on 20 of 20 seeds in every cell, so the ratio is a pure implementation speedup measured pairwise. The fifth column additionally widens the candidate neighbourhood, which changes the search: it reaches the target in 421 instead of 676 iterations on $K_6(8, 4)$ and 23 instead of 30 on $K_8(9, 4)$, but costs a factor 3.6 on the small tight cell.

Caching move evaluations between iterations is impossible here. A cached value for codeword c survives a commit on c' only if their evaluation spheres are disjoint, which needs $d(c, c') > 2R + 2$; but $2R + 2 \geq n$ for every cell of interest, so every commit invalidates every entry. Large R relative to n — the defining feature of the cells whose bounds are inherited arithmetic — is exactly what rules caching out.

Early exit on a lower bound. During a candidate’s walk the final change is bounded below using the fact that the entering count cannot exceed $|U|$. The bound is exact and prunes 1.03%, 0.006% and 0.000% of patterns on the three cells: it only becomes strong as $|U|$ falls, and small $|U|$ is precisely what makes the sparse evaluator fast instead.

Inverting the leaving side, i.e. maintaining a list of the singly covered words as we do for the uncovered ones, pays only if $n|U_1| < \binom{n}{R}(q-1)^R$. Measured on the recorded codes, $|U_1|$ is 5357, 6592 and $\sim 10^6$ against thresholds of 417, 5469 and 33 614: it fails everywhere. Uncovered words are rare by construction, singly covered words are not.

Symmetry reduction. Translating the code so that one codeword sits at 0^n and freezing it quotients out the q^n translations, but gives no measurable benefit ($4.87\times$ against $5.26\times$ on $K_6(8, 4)$, $0.89\times$ against $2.11\times$ on $K_6(6, 3)$). A local search does not revisit states exactly, so removing a symmetry removes almost no work; all it does here is shrink the candidate pool by one codeword, which on the small cell measurably hurts.

Deterministic focus on the longest-uncovered word, the cheapest form of a guided objective,

is clearly harmful: $2.04\times$ against $5.26\times$ on $K_6(8, 4)$, and on $K_6(6, 3)$ only 5 of 20 runs reached the target at all. Always choosing the same word destroys the randomisation that lets focused local search leave a plateau. A stochastic age weighting remains untested.

Software prefetching and *transparent huge pages* are both within noise. For huge pages the reason is architectural: the base page on this host is already 64 kB, so even the 268 MB counter array spans only 4096 pages.

Restart policy

The empirical run-time distributions answer the restart question directly. At targets the search reaches routinely the distribution is light-tailed (coefficient of variation 0.20–0.47 on the two loose cells) and a fixed restart interval cannot help; calibrating the best observed gain against the memoryless null gives $p = 1.0$. On the tight cell $K_6(6, 3)$ at a target hard enough that a quarter of runs stall, the distribution becomes heavier than exponential and restarting every ≈ 1.6 CPU seconds cuts the expected time to target by a factor of 4.2 ($p = 0.002$). The policy that follows is to run one chain while it is still improving and to rely on independent parallel workers rather than on timed restarts.

4 Upper bounds: structure, and the production portfolio

4.1 Structural solvers

Free search over subsets of $\mathbb{Z}_q^n$ is the wrong tool for cells whose optima are algebraic, and several of Kéri’s incumbents are exactly that. Two structural solvers close the gap.

Unions of cosets of a linear code. When q is a prime power and $M = cq^k$, we search for codes of the form $C = \bigcup_{i=1}^c (x_i + L)$ with L a linear $[n, k]_q$ code. Covering is then a property of syndromes: C covers $\mathbb{Z}_q^n$ iff the c syndromes of the x_i cover the q^{n-k} -element syndrome space under the weighted distance induced by coset-leader weights, collapsing a q^n -word problem to a q^{n-k} -syndrome one. The effect is drastic on the cells it fits: the incumbent $K_8(8, 4) \leq 512 = 8^3$ — which free search could not even reproduce (its best from scratch was 740) — is reconstructed and verified in 0.18 seconds including process start-up, and the descent to the new record 505 then starts from a verified incumbent. The same solver settles $K_5(11, 5) \leq 625 = 5^4$ and the known-hard $K_3(11, 4) \leq 81 = 3^4$ (a cell on which the free search plateaued 410 words short) in under a second each.

Translation quotients. A second solver searches codes invariant under $x \mapsto x + \mathbf{1}$, working in the quotient (size q^{n-1} effective) with a genuinely free remainder: measurements on $K_6(8, 4)$ showed that neither a fully invariant code (which provably plateaus, since one spare codeword cannot repair a deficient orbit) nor a fully free search solves the cell, while “mostly invariant plus a free remainder” solves it reliably. On the largest cells the quotient search beats the best free search by 17% at equal core-seconds.

4.2 A solver competition with frozen rules

The production configuration was selected by a competition rather than by intuition. Four solver lineages — the algorithm-engineering line of Section 3.2, a low-level SIMD line, the structural line of Section 4.1, and a portfolio-scheduling line — were developed independently against a published contract: a fixed entry-point interface, ≤ 6 threads, ≤ 25 GB, frozen scoring (a full

verified cover scores 1000; a partial cover scores the negated uncovered fraction; invalid output scores worse than any valid partial), public development cells, and *held-out* judging cells revealed only at evaluation time, 15 seeds each, all entries sharing one machine in one window. The judge verifies every output file independently; no solver’s self-reported count is ever trusted. One scoring amendment was made before judging (normalising partial scores by q^n) after a competitor demonstrated that the original absolute penalty let invalid output outscore valid partial covers on large cells.

The outcome was not a single winner but a decomposition: the structural line won the held-out judging outright (11/28 solves), the engineering line had the fastest evaluator, the low-level line had the fastest inner loop on cache-resident cells and the only deadline-safe initialisation and atomic-publishing discipline, and the portfolio line’s scheduling (many single-threaded chains, stop on first solve) beat multi-threading everywhere except the largest cells. The production engine is the merge of all four, each component verbatim where it won:

- **Phase A (algebra, seconds):** the coset solver over every feasible k ; seeds from recorded codes and from direct sums over every split, taken from the growing result set.
- **Phase B (portfolio):** independent single-threaded chains with distinct engines, seeds, and symmetry settings; the mix decided by measured thresholds (q^n , redundancy $q^n/(M|B_R|)$, initialisation cost against budget). Below $3 \cdot 10^5$ words half the chains run the SIMD engine; above $5 \cdot 10^7$ the portfolio is all-quotient; the wide neighbourhood is enabled near redundancy ≥ 3 ; timed restarts only on tight cells.
- **Discipline:** the wall clock covers initialisation; the incumbent is published atomically on every improvement, so a kill at any moment leaves the best code on disk; a finishing pass re-reads the winner from disk, de-duplicates, and recomputes coverage by direct ball marking.

Under the frozen judge the merged engine was at least as good as every component on every cell and strictly better on three of four public cells (14/20 solves against the best component’s 12/20, and 2.2–2.9 $\times$ faster to a cover where both solve). All record cells were then attacked with the merged engine under a descend-until-wall protocol.

4.3 Hardware, and a controlled CPU/GPU comparison

All eighteen record codes were found on the 64 Neoverse-V2 cores of a single NVIDIA GH200 node; per-cell effort ranged from seconds to a few core-days. Because the promise of GPU acceleration for this problem family is a natural question, we report a controlled comparison on the same node (a 480 GB GH200, H100-class GPU).

Two GPU formulations were implemented and verified. The first casts candidate evaluation as small integer matrix products over coverage bitmaps to engage the tensor cores; it is structurally starved, because the contracted dimension is only $nq \approx 50$ (30 $\times$ below the arithmetic-intensity knee of the hardware), and never approached the CPU. The second runs 256 independent search chains in one persistent kernel (one chain per thread block, no host synchronisation, the whole counter state resident in HBM). At equal validated search semantics, the persistent kernel reaches 0.09 $\times$ the throughput of the full 64-core socket on $K_6(6, 3)$, 0.10 $\times$ on $K_6(8, 4)$, and 1.24 $\times$ on the largest benchmarked cell $K_8(9, 4)$ ($q^n = 1.3 \cdot 10^8$, aggregate 1492 versus 1200 moves/s) — the regime where each chain’s working set defeats the CPU caches while the GPU’s

bandwidth advantage applies. The honest summary is that on this problem a GPU currently buys about one extra CPU socket of capacity on the very largest cells and is counterproductive below $q^n \sim 10^8$; all bounds reported here are CPU results.

5 Lower bounds: exact certificates for $q \geq 6$

5.1 The relaxation

Gijswijt and Polak [3] bound $K_q(n, R)^3$ below by a semidefinite program over variables indexed by orbits of triples of words under $\text{Aut}(q, n) = S_q \wr S_n$, with positive-semidefiniteness constraints from the block-diagonalisation of the Terwilliger algebra of the Hamming scheme, linear constraints from sphere-covering inequalities, and matrix-cut constraints (their Theorem 4.18). Two facts make the $q \geq 6$ corner attackable. First, the size of the reduced SDP depends only on n — at $n = 11$ it has 339 variables and 126 blocks of size at most 12 — so alphabet size costs nothing; q enters only through the coefficients. Second, published work stops at $q \leq 5$, so the incumbents at $q \geq 6$ are the 2011 table values.

We reimplemented the reduced SDP from scratch with the entire model generation in exact integer arithmetic, and validated the implementation four ways: (a) a self-test that pushes explicit verified covering codes through the model and checks every constraint in exact rational arithmetic (objective exactly $|C|^3$); (b) a coefficient-level comparison against the authors’ published Julia code, matching every generated coefficient — orbit partitions, objective, block-diagonalisation, distribution numbers — over six (q, n, R) cells spanning $q = 3$ to 7, tens of thousands of big-integer values each; (c) reproduction of their published values: over the 49 published table values we recomputed, 44 match to published display precision, and 18 of the 22 published *improved lower bounds* we attempted are reproduced exactly at the integer level (the misses are all in precision-limited territory, below); (d) the invariant that no certified lower bound may exceed a known upper bound, checked against all 1145 table cells — no violation ever occurred.

5.2 Certification: nothing floating-point is trusted

The SDP is solved numerically (an interior-point method; a first-order solver as fallback), but the solver’s output is used only as a *suggestion*. By weak duality, any feasible point of the dual — multipliers $y_k \geq 0$ for the linear constraints and PSD matrices Y_b for the blocks — certifies the bound it attains, regardless of how it was found. We therefore round the numerical dual to rationals over a dyadic denominator (2^{46}), push each Y_b strictly into the PSD cone, and, if any dual inequality $d_v \leq c_v$ fails in exact arithmetic, shrink the whole dual by a dyadic factor θ , which restores exact feasibility at the cost of scaling the bound. A well-conditioned solve gives $\theta = 1$.

The resulting certificate — the rationals y_k , Y_b and the claimed value — is checked by a standalone program of about seven hundred lines that uses only the Python standard library: it *re-derives the entire SDP* from (q, n, R) in exact integer arithmetic, checks $y_k \geq 0$, checks each $Y_b \succeq 0$ by an exact symmetric-pivot LDL^T factorisation over rationals, checks every dual inequality in integer arithmetic, and takes the cube root by exact integer comparison. It also checks that the covering inequality supplied in the certificate is itself valid, so a certificate cannot smuggle in an unsound constraint; tampered certificates are rejected. The seven bounds of Table 2 are precisely the cells (out of 163 certified) where the certified value exceeds the 2011

incumbent. Every number in Table 2 rests on this checker alone; the floating-point solve is not part of the trusted base.

5.3 Conditioning, and where the improvements land

Making the interior-point method survive these instances was the main technical obstacle: the optimum is $|C|^3 \sim 10^{13}$ while every constraint constant is $O(1)$, and the objective coefficients span ten orders of magnitude, so a naive scaling collapses on exactly the interesting cells (the solver reports spurious infeasibility, or the rounded dual loses half its value to θ). The scaling that works is a geometric-mean equilibration, applied entirely in powers of two so that the map back to the unscaled dual is exact: normalise the objective by the cube of the sphere-covering bound and give each variable the magnitude profile of the square root of its objective contribution. On $K_6(8, 3)$ this lifts the certified value from 183 to 240 against a true SDP optimum of 239.52; on $K_6(10, 4)$ from 56 to 441.

Even so, double precision carries the method only to SDP values around $1.5 \cdot 10^9$ (cube root ≈ 1150); beyond that the solver fails in a way the pipeline detects automatically (the SDP provably dominates the sphere-covering bound, so any certificate below it flags a failed solve rather than a weak bound — such cells are reported as failures, never as bounds). This ceiling, not the model, is what limits the present sweep: 68 of the 163 certified cells are in the failed-solve state, concentrated on the large- $R = 2$ cells where the incumbents are barely above sphere-covering and the head-room is greatest, and several failures are near-misses ($K_{20}(8, 4)$ certified 2615.7 against an incumbent 2760; $K_{21}(8, 4)$ 3135.1 against 3288). A higher-precision solver (the published $q \leq 5$ results used 512-bit arithmetic) would open that range; we did not have one for this hardware.

The pattern of the seven successes is exactly what this analysis predicts: every improved cell has an incumbent within 1–4% of the sphere-covering bound (Kéri keys **y** and **s**), while no cell with a Haas–Halupczok–Schlage–Puchta incumbent (key **m**, 13–52% above sphere-covering) is improved — at $q \geq 6$ this level of the SDP hierarchy delivers 1.15–1.43 $\times$ sphere-covering, decaying with q , and so overtakes the combinatorial bounds only where they are weak. We also note that the fractional-covering *linear* program cannot help at all here: since $\text{Aut}(q, n)$ acts transitively on $\mathbb{Z}_q^n$, averaging shows its optimum is exactly $q^n/|B_R|$ — the LP *is* the sphere-covering bound, which is why the track goes straight to the SDP.

6 Verification

False records are worse than no records, so verification is exhaustive, exact, and redundant, and no solver’s internal state is ever the evidence.

Upper bounds. Every claimed code, re-read from its published file, was checked by four methods, two of which share no code with the other two: (1) ball marking: mark all words within distance R of each codeword in a q^n bitmap and confirm the bitmap is full; (2) meet-in-the-middle bitset covering over a coordinate split; (3) a min-distance scan over all q^n words, which also reports the exact covering radius; (4) Hamming-graph dilation: R -fold neighbourhood dilation of the code’s indicator array, computing no distances at all, with cost independent of M . In addition, a fifth, independently written dilation verifier (produced without access to the other implementations) re-confirmed all eighteen codes from the published files immediately before this version was prepared. The largest check ($q^n = 6^{10} = 60\,466\,176$ words) completes in seconds. The distinct-codeword count of every file is checked against the claimed M .

Lower bounds. Section 5.2: exact rational dual certificates, an exact standalone checker, and the global invariant against known upper bounds. The seven record certificates were re-verified immediately before this version was prepared.

The ancillary files include `verify_cov.py`, a standalone dependency-free verifier (`python3 verify_cov.py CODEFILE q n R` re-checks any claimed upper bound in at most a few minutes; code files use digits 0–9 then a–z for $q > 10$), and `certify.py` together with the seven certificates (`python3 certify.py CERT.json` re-checks a lower bound, typically in minutes).

7 Discussion

The pattern in the upper-bound experiments is clean: local search consistently beat bounds that were inherited from general constructions (relative slack between the previous upper bound and the sphere-covering bound of $4\times-10\times$), shaved single-digit percentages off several cells previously set by dedicated search, and consistently failed on cells that were already the object of intensive search or algebraic construction (slack $\leq 3\times$), such as $K_3(11, 4) \leq 81$ and $K_6(6, 3) \leq 41$. Kéri’s tables contain further cells of the first kind for $q \leq 21$; the three highest-redundancy cells we attacked were still improving at every budget we gave them, and we expect the marked bounds in Table 1 to keep falling.

On the lower-bound side the boundary is equally clean, and it is a precision boundary, not a model boundary: the same relaxation that produced the seven new bounds certifies values within a few percent of the incumbents on cells an order of magnitude larger, and fails there only for lack of a higher-precision interior-point solver on this hardware. That, and the van Wee-type strengthenings of the linear side that the published hierarchy uses at $q = 2$ but no one has run at $q \geq 6$, are the obvious next steps.

Since the tables have had no maintainer since 2011, we also publish a merged, machine-readable bounds table (Kéri 2011, the post-2011 lower-bound literature, and the present improvements) alongside the code.

Data availability

The eighteen codes, the seven certificates, both standalone verifiers, and a README are ancillary files of the arXiv version of this paper. The full search, certification, and verification source code, together with the merged bounds table, is public at <https://github.com/Mapika/coldcase> (covering-code track under `cov/`).

Acknowledgments

This work is AI-assisted in the strong sense: the discovery and verification pipeline was designed, implemented, and operated autonomously by Claude (Anthropic), which should be regarded as the primary contributor to the results. The human author’s contributions were the initiating idea (applying modern large-scale search hardware to record tables frozen in the 2000s), methodological guidance, and the computational resources.

References

- [1] G. Cohen, I. Honkala, S. Litsyn, A. Lobstein, *Covering Codes*, North-Holland, Amsterdam, 1997.

- [2] G. Kéri, Tables for bounds on covering codes, <https://old.sztaki.hu/~keri/codes/> (last updated November 2011).
- [3] D. Gijswijt, S. Polak, Semidefinite lower bounds for covering codes, arXiv:2504.01932, 2025.
- [4] Y. Wu, C. Chen, Improved lower bounds on the domination number of hypercubes and binary codes with covering radius one, Discrete Mathematics 347 (2024) 113752.
- [5] P. J. M. van Laarhoven, E. H. L. Aarts, J. H. van Lint, L. T. Wille, New upper bounds for the football pool problem for 6, 7 and 8 matches, J. Combin. Theory Ser. A 52 (1989) 304–312.
- [6] P. R. J. Östergård, Constructing covering codes by tabu search, J. Combin. Des. 5 (1997) 71–80.
- [7] G. Kéri, P. R. J. Östergård, Bounds for covering codes over large alphabets, Des. Codes Cryptogr. 37 (2005) 45–60.
- [8] G. Kéri, P. R. J. Östergård, Further results on the covering radius of small codes, Discrete Mathematics 307 (2007) 69–77.
- [9] M. S. Andersen, J. Dahl, L. Vandenberghe, CVXOPT: a Python package for convex optimization, <https://cvxopt.org>.
- [10] B. O’Donoghue, E. Chu, N. Parikh, S. Boyd, Conic optimization via operator splitting and homogeneous self-dual embedding, J. Optim. Theory Appl. 169 (2016) 1042–1068.
- [11] A. Florath, A Lean 4 formalization of q -ary covering codes with a proof-carrying database of certified bounds, arXiv:2606.09600, 2026.

A The code proving $K_6(8, 4) \leq 166$

For self-containedness we print the largest single improvement ($216 \rightarrow 166$, -23%). Each row lists codewords of $\mathbb{Z}_6^8$; the machine-readable version is the ancillary file `K6_8_4_M166.txt`.

```

01430421 12541532 23052043 30103154
45214205 50325310 03405225 14510330
25021441 30132552 41243003 52354114
03134001 14245112 25350223 30401334
51512445 52010550 02442243 13553304
24004415 35115520 40220031 51331142
02303231 13414342 24525453 35030504
40141015 51252120 05143544 10254055
21305100 32410211 43521322 54032433
04301535 13432040 20543151 31054202
42105313 53210434 04043240 15154351
20205402 31310513 42421024 53532135
05111133 10222244 21333355 32444400
43555511 54000022 01045452 12150503
23201014 34312125 45423230 50534341
00544125 11055230 22100341 33211452
44322503 55433014 02231500 13342011
24453122 35504233 40015344 51120455
05223315 10334420 21445531 32550042
43001153 54112204 03411050 14542101
25033212 30144323 41255434 52300445

```

04450305	15501410	20012521	31123032
42234143	53345254	00052413	11103524
22214035	33324140	44430251	55541302
02115012	13220123	24331234	35442345
40553450	51004501	01024314	12135425
23240530	34351041	45402152	50513203
05352332	10413443	21514554	52025505
43130110	54241221	04205542	15315053
20420104	31531215	42042320	53153431
01213111	12324222	23435333	34540444
45051555	42131403	00300252	11411303
22522414	33033525	44104030	55255141
05535024	10040135	21151240	32202351
44313402	54424513	01500043	12051134
23122205	34233310	45344421	50455532
03314404	10402515	25530020	31021031
41132242	52243353	12005001	03402520
31455113	23344502	55203541	32515205
30325000	02023150	14021545	54142054
34124521	03523252	45250342	00522424
44553234	53044110		